

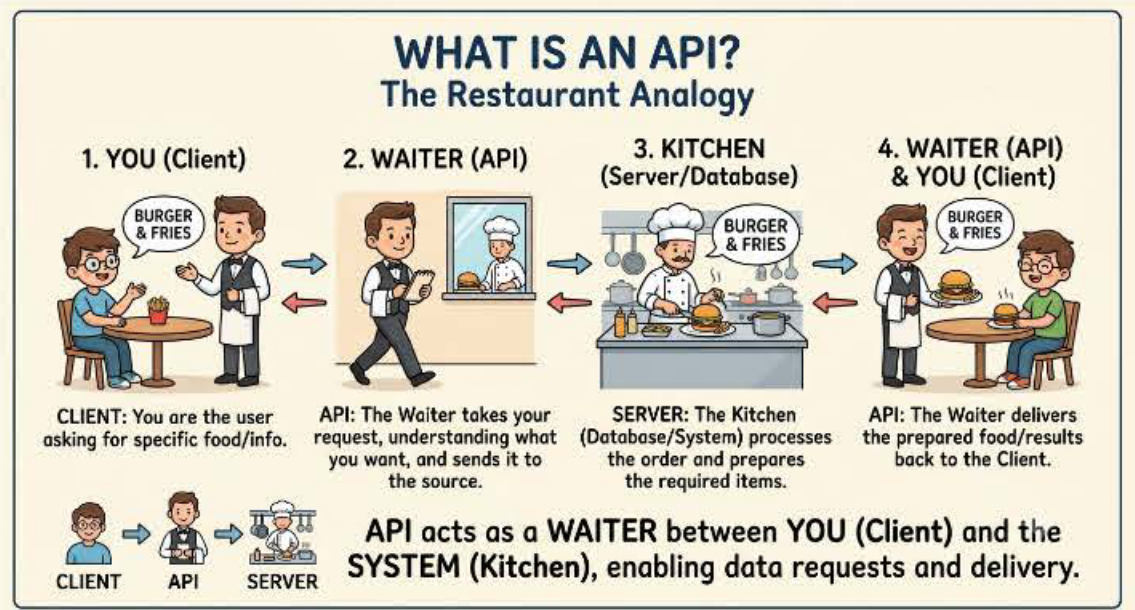
|- Why can't we use the JEE to perform the same thing which can be done by the Spring framework ?

-> JEE is stand for Java Enterprise Edition and it is an API

An API is a way for two applications to talk to each other.

API (Application Programming Interface) is like a messenger that takes your request to a system and brings back the response.

Simple Example
=====



|- Api are provided by the languages . so that developer can develop application using the language provided classes easily.

|- Java standard API , there api's contain only interface/abstract classes , they don't provide the concrete classes at all
The implimentation of the api will have the concrete classes
Api's are incomplete or you can say it as partial

Incomplete vs Complete in Java

Interface & Abstract Class

- |- Do not provide full implementation
- |- Contain only method declarations (or partial code)
- |- Called incomplete / partial

```
Interface
interface Animal {
    void sound(); // method declaration (no body)
}
```

```
Abstract class
abstract class Animal {
    void sound(); // method declaration
    void eat(){ // concrete body
}
```

Concrete Class

- |- Provides full implementation of methods
- |- Can be directly used to create objects
- |- Called complete

Example

```
class Dog implements Animal {

    public void sound() {
        System.out.println("Dog barks");
    }

    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
    }
}
```

1. Api's are generally huge in nature . It provide a lot of classes and interface and the most important things is all the classes in the api are inter- dependent in nature mean they are depended on each other

For Example
JDBC Api

```
Connection con=DriverManager.getConnection(url,username,password);
Statement st=con.createStatement()/con.preparedStatement();
ResultSet rs=st.executeQuery()/ st.executeUpdate();
```

- |- Some of them are related and work together, but not all are dependent on each other.

For Example :

java.util
List, ArrayList, Map → related

java.io
File, InputStream → different purpose

2. Let's Take the example of Jdbc api

```
Class.forName("oracle.jdbc.driver.OracleDriver");
Connection con=DriverManager.getConnection(url,username, password);
Statement st=con.createStatement()/con.preparedStatement();
Result rs= st.executeQuery("select * from emp");
while(rs.next())
{

}
```

Suppose in the same manner we have to perform the insert , update ,delete and fetch operation

1. Load the driver
2. Create the connection
3. Create the statement
4. Execute the query
5. get the result set
6. iterate over the resultset

[Common requirement]

- |- In every project and for every functionality if i have to write the same repeated code and logic that particular logic is known as " boiler plate logic ".

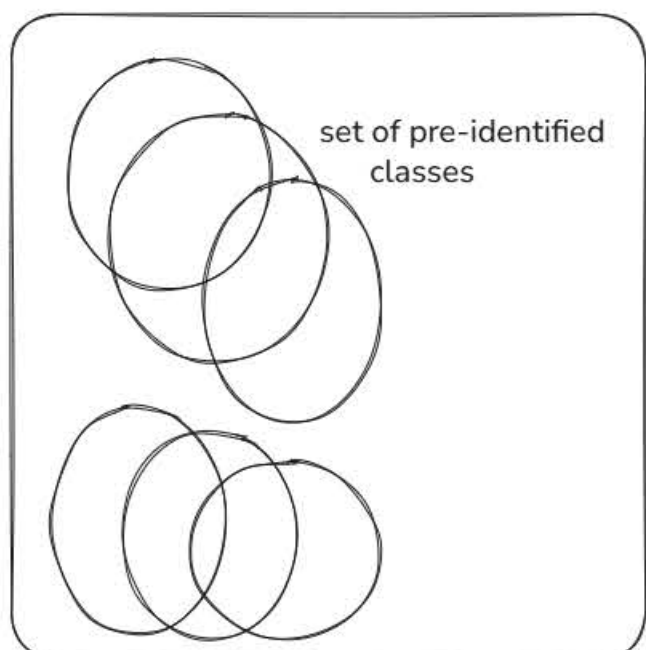
3. Api's not support rapid or fast application development the reason is one cannot learn api partial and cannot and cannot able to build an application due to the inter dependency of the classes

Api's doesn't provide the solution of writing the repeated set of logic which we can say it as boiler plate logic . As a result below points will create the issue

1. Programmer have to write same set of code in every functionality.
2. The effort and the time required for building the application will become high.
3. For buiding the application more man power needed.
4. Cost for creating the application will increased.
5. More lines of code can increase the chances of bugs.
6. More lines of code will effect in testing.

What is the Framework

Framework also provide the bunch of classes .These classes will have the pre-identified functionality means the solution of the repeated set of code or boiler plate logic which we want to use in developing our application



```
class SqlQueryTerms {

    public void getResult( )
    {
        Connection con=
            DriverManager.getCon(url,
                username,pwd)
        Statement st=con.createStmnt();
        ResultSet rs=st.executeQuery();
    }
}
```

//We are using it by creating the objecy

```
SqlQueryTerms logic=new SqlQueryTerms();
logic. getResult();
```

all the classes provided by framework are concrete classes and there are no inter-dependency between classes. This makes framework more easy to use and learn as well.

When compared with APIs framework has less no of classes. So it is quick to learn. People can learn framework partially and can start developing applications.

As we know Framework provides boiler-plate logic from the above, so there are many advantages here.

1. efforts and time required in developing the application will be less.
2. less developers are required to develop application.
3. obviously less cost in developing application.
4. chances of increasing the bugs are very less.
5. Time required for testing the application is very less.

This makes the use of Framework for rapid application development.